An Exploratory Comparison of Static Harness Construction for Cedar Policy Synthesis

Anonymous ACL submission

Abstract

How should reusable instructions for policy synthesis be constructed? We compare a fixed baseline (Manual), direct generation (Direct), guidance followed by generation (Meta), and generation followed by one revision (Refine). Each method supplies one static harness to the same Codex runtime on 20 shared Cedar validation scenarios. Success rates are 60%, 50%, 45%, and 55%, respectively. Meta minus Refine is -10 percentage points, with a post-hoc paired bootstrap 95% interval of $[-30, +10]$. Case analysis identifies target-language type errors and incorrect authorization conditions. These preliminary observations do not establish a general ranking: construction variance is unmeasured, validation is small, and historical solver-log gaps limit traceability.

1 Introduction and Related Work

Translating access requirements into executable policies requires both valid syntax and correct decisions about who may access which resources. Cedar supports schema-based validation and formal policy analysis (Cutler et al., 2024), making it possible to distinguish language errors from authorization errors.

This project studies the construction of a *static harness*: a structured instruction artifact reused across synthesis tasks. Guidance-first construction separates proposing a translation strategy from writing the final instructions; revision starts with a concrete instruction draft. Our research question is whether these two construction procedures yield different validation outcomes under a fixed runtime. A fixed baseline and direct generation provide context for this comparison.

OPRO generates and evaluates candidate instructions in an iterative optimization process (Yang et al., 2024). Self-Refine iteratively uses model-generated feedback to revise outputs (Madaan et al., 2023). Our comparison uses fixed construction stages without intermediate candidate evaluation or search. Refine revises the reusable harness once, rather than repairing each generated policy. RAISE supplies Cedar synthesis materials and executable verification plans and studies training with formal feedback (Zhou et al., 2026). We reuse its task materials for an inference-time pilot, without model training, online adaptation, or a claim to reproduce its benchmark results.

Split	Size	Role
Design	20	Baseline error analysis
Validation	20	Shared method comparison
Held-out	375	Reserved; unopened

Table 1: Pilot split roles. Design and validation are new grouped subsets of the release's non-held-out data.

2 Experimental Design

2.1 Task and data

Each generation session receives natural-language requirements, a Cedar schema, a fixed harness, and common output instructions, and returns one policy in a structured response. Reference policies and verification plans are withheld from the model. Tool use, verifier feedback, answer repair, and best-of-N selection are excluded.

From a pinned upstream release, non-held-out scenarios are grouped by family and normalized schema/reference-policy signatures. We shuffle 139 connected groups with seed 20261110 and sample one scenario from each of the first 40 groups: 20 for design and 20 for validation. These sets have disjoint scenario IDs, families, and structural signatures. All four methods use the same validation scenarios. The 375 held-out scenarios remain unopened and unevaluated; their count comes from metadata.

Method	Process	Stages	Attempts
Manual	Fixed baseline	0	0
Direct	Packet → harness	1	1
Meta	Guidance → harness	2	3
Refine	Draft → revision	2	2

Table 2: Construction stages and attempts. Meta includes one timeout and recovery; Manual is an AI-assisted fixed baseline.

Method	Pass	Rate	Syn.	Type	Sem.
Manual	12/20	60%	0	0	8
Direct	10/20	50%	0	2	8
Meta	9/20	45%	0	4	7
Refine	11/20	55%	0	1	8

Table 3: Validation success and failure counts. Syn.: syntax; Type: schema/type; Sem.: semantic. Each method retains all 20 scenarios.

2.2 Harness construction

Table 2 summarizes the conditions. Manual is the historical label for a fixed baseline prepared in an AI-assisted workflow; independent human-only authorship is not established. A shared frozen construction packet contains three fixed design inputs, the baseline harness, and a summary of its design-set errors, but no reference policies or verification plans. Each automatic method produces one final JSON harness under a 500-word limit.

Direct maps the packet to a harness. Meta first produces design guidance, then generates a harness in a fresh session using that guidance. Refine first generates a draft harness, then revises it once in a fresh session. No intermediate artifact is evaluated. There is only one construction repeat per method.

2.3 Runtime and evaluation

The recorded runtime is official Codex CLI 0.162.0-alpha.17.2 via a ChatGPT subscription, requesting `gpt-6.1-sol` with medium reasoning effort. The actual serving model is unknown. Fresh policy sessions run serially with method order rotated across scenarios. Matching stage counts and request settings do not guarantee equal compute.

Evaluation proceeds through syntax parsing, schema/type validation, and the saved plan's nonempty set of semantic checks. The primary binary outcome requires all gates to pass. The verifier uses Cedar 4.14.1 (experimental) and cvc5 1.4.2 (Barbosa et al., 2022). Floor checks detect missing required permissions, ceiling checks detect excess permissions, and liveness checks test whether specified behavior is possible. Thus, success means compliance with the saved plan, not textual similarity or correctness under every interpretation of the prose. Type failures remain in the primary denominator; their unexecuted semantic checks are unknown.

3 Progress and Preliminary Results

3.1 Completed work and deviations

The completed E0–E5 sequence covers runtime diagnostics, verifier controls, baseline development, harness construction, validation, and evidence analysis. All 18 verifier controls met expectations: five reference policies passed, ten semantic mutations were detected, and three syntax/schema errors were classified. Manual passed 10/20 design scenarios; this development result is separate from validation.

The ledger records 110 logical model-call attempts against a cap of 120: three diagnostics, 20 development-policy attempts, six construction attempts, and 81 validation-policy attempts. There are 108 completed attempts, 100 final policies, and five logical automatic-construction stages. Internal service request counts are unknown.

Two attempts returned no final response: Meta's final construction stage and one Refine validation attempt. Each received one same-input, fresh-session retry under a uniform infrastructure-recovery rule. The initial zero-wrapper-retry protocol was amended after the first failure and before validation; this disclosed deviation was not preregistered. Completed wrong answers were never retried. Both failed attempts lack token usage, leaving total token cost unknown. Although the configured timeout was 180 seconds, recorded wall times were approximately 180 and 1165.7 seconds. The latter discrepancy is unexplained; wall latency is not a compute measurement.

3.2 Validation outcomes and uncertainty

Table 3 includes all 80 final outputs on 20 shared scenarios. Failure categories identify the first failed gate. No final evaluation remains unresolved; failed infrastructure attempts are retained separately in the ledger.

Meta minus Refine is $9/20 - 11/20 = -10$ percentage points. Table 4 shows eight joint passes,

	Refine pass	Refine fail
Meta pass	8	1
Meta fail	3	8

Table 4: Paired outcomes for the same 20 scenarios.

one Meta-only pass, three Refine-only passes, and eight joint failures. A post-hoc paired percentile bootstrap gives a 95% interval of $[-30, +10]$ points. We resample 20 scenarios with replacement, retaining both outcomes, for 100,000 replicates using Python's MT19937 generator, seed 20261111, and linearly interpolated percentile endpoints. Pairing preserves the shared evaluation units (Koehn, 2004).

This interval conditions on the two fixed harnesses and saved outputs; it excludes construction and repeated-generation variance. The 80 outputs are not 80 independent scenarios. The interval includes zero and establishes neither superiority nor equivalence. Manual's highest observed success rate likewise does not establish a general advantage.

4 Error Analysis

Original policies, validator messages, and solver logs were inspected for all four Meta–Refine discordances. The three Refine-only cases (Kubernetes, IoT dispatch, and immigration clearance) are Meta type failures; the Meta-only case is a Refine semantic failure. The following two cases were selected post hoc for exposition. No sample, policy, or score was changed.

Kubernetes token expiry. The requirements prohibit reading a secret with an expired token; both current time and expiry are `datetime` values. Meta calls `lessThan` and `greaterThanOrEqual` on them. The validator exits with code 3 and reports *expected decimal but saw datetime*, preventing semantic evaluation. Refine uses `>=` in the expired-token prohibition and passes validation and all seven semantic checks. This is a concrete type/API error; it does not establish that repairing that expression would make Meta pass all checks or that a particular harness sentence caused the error.

Grant obligation. The grant workflow requires prior approval by someone other than the obligating grants officer. Refine permits a program officer to obligate funds, forbids requesters without the grants-officer role, and applies the same-approver veto only if optional `approvedBy` exists. Both policies pass type validation. A floor counterexample has a requester with only the grants-officer role and a different approver: Refine incorrectly denies it. A ceiling counterexample has both roles but no `approvedBy`: Refine incorrectly permits it. Meta requires the grants-officer role and a present approver distinct from the requester; all five checks pass. This identifies a local role/approval-state failure under the plan, not general Meta superiority.

5 Limitations and Next Steps

Experimental scope. Twenty validation scenarios and one construction repeat per method cannot characterize general performance or construction variance. Stage counts, actual attempts, and token costs are different quantities. Runtime context and serving-model identity are only partially observable. Requirements and verification plans may also diverge: a development patent scenario's plan requires examiner assignment that its permission sentences do not explicitly require. Plan compliance therefore has specification-validity limits. Analysis revision a2 corrected a1's description of empty semantic-check lists after type failure; predictions and scores were unchanged.

Evidence coverage. Saved results can be recomputed, and current exported validator/solver decisions agree with all 80 scores. Full historical byte-level traceability is incomplete: 477 of 538 semantic-check logs retain their recorded hashes, while 61 do not, affecting nine results. Executions reused verification-log directories, so later runs overwrote earlier bytes. All four discordant cases retain their key evidence, and all 110 model-event streams match recorded hashes. Nevertheless, the historical audit returns `INCOMPLETE`. Current logs cannot recover missing originals; no rerun is presented as restoration.

Next steps, in priority order. First, prevent log overwrites with immutable per-attempt directories. Second, independently repeat harness construction to estimate its variance. Third, expand validation with additional isolated groups. Fourth, test identical Cedar type/operator guidance across all methods as a controlled diagnostic. Fifth, evaluate held-out data only after freezing the final system and analysis. These are future experiments.

References

Haniel Barbosa, Clark W. Barrett, Martin Brain, Gereon Kremer, Hanna Lachnitt, Makai Mann, Abdalrhman Mohamed, Mudathir Mohamed, Aina Niemetz, Andres Nötzli, Alex Ozdemir, Mathias Preiner, Andrew Reynolds, Ying Sheng, Cesare Tinelli, and Yoni Zohar. 2022. cvc5: A versatile and industrial-strength SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS), Part I*, volume 13243 of *Lecture Notes in Computer Science*, pages 415–442. Springer.

Joseph W. Cutler, Craig Disselkoen, Aaron Eline, Shaobo He, Kyle Headley, Michael Hicks, Kesha Hietala, Eleftherios Ioannidis, John Kastner, Anwar Mamat, Darin McAdams, Matt McCutchen, Neha Rungta, Emina Torlak, and Andrew Wells. 2024. Cedar: A new language for expressive, fast, safe, and analyzable authorization (extended version). *Preprint*, arXiv:2403.04651. Version 2.

Philipp Koehn. 2004. Statistical significance tests for machine translation evaluation. In *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing*, pages 388–395, Barcelona, Spain. Association for Computational Linguistics.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024. Large language models as optimizers. In *International Conference on Learning Representations*.

Yingming Zhou, Adarsh Vatsa, and William Eiers. 2026. RAISE: Reinforcing access control policy synthesis in LLMs via symbolic evaluation. *Preprint*, arXiv:2609.33796. Version 1; preprint.